弦烛之声 · XIANZHU DSH CONNECT

新手教程

在自己电脑上跑一个 AI 模型

从零开始，一步步把这个 App 用起来。
不需要懂编程，不需要命令行基础。

> 你会得到什么？
>
> 一个装在你 Mac 上的 AI 模型。它不联网、不收费、不把你的对话传到任何地方。你可以在 DeepSeek Harness 里像平常一样跟它聊天、让它帮你干活——只不过这次，全部运算都发生在你自己的电脑里。

这份教程怎么读

从头读到尾大概花 20 分钟。如果你想马上用起来，直接跳到第 3 章跟着做，遇到不懂的地方再回头翻前面的章节。

目录

第 1 章 先认识几个词 用大白话解释模型、GGUF、token 这些名字

第 2 章 准备工作 确认你的电脑能跑，把模型文件下好

第 3 章 五步上手 导入、启动、选用——完整走一遍

第 4 章 界面逐个说 每个按钮是干什么的

第 5 章 为什么会「装不下」 理解上下文和提示词成本

第 6 章 把速度调快 几个让模型跑得更顺手的设置

第 7 章 遇到问题怎么办 常见毛病与对策

附录 文件都在哪 配置与日志的位置

第 1 章

先认识几个词

这一章没有操作，只是把后面会用到的词讲清楚。如果你已经知道这些，可以跳过。

模型：一个装在硬盘上的「大脑文件」

平时说的「GPT」「通义千问」这些 AI，本质是一个巨大的数学公式，里面存着几千亿个数字。这些数字是它读了海量文字之后「学」出来的。

当你把它下载到电脑上，它就是硬盘里的一个文件——可能 20 GB，也可能 40 GB。运行时，这些数字被读进内存，你的每一句话都会被「算」一遍，然后吐出回答。

> 打个比方：模型文件像一本特别厚的菜谱。
> 推理就是照着菜谱做菜的过程——菜谱本身不会做菜，得有人（你的电脑）把它读一遍、动手做。

GGUF：模型文件的一种格式

模型文件有好几种格式，这个 App 用的是 GGUF。它的特点是压缩过、能直接在普通电脑上跑，不用显卡服务器。

你在网上看到的文件名通常长这样：

```
Qwen3.6-35B-A3B-Q4_K_P.gguf
```

拆开看是这样：

部分	意思
Qwen3.6	模型的名字，就像手机的型号
35B	参数规模：350 亿个数字。越大越聪明，也越吃内存
A3B	说明它工作时只激活一部分，速度快很多
Q4	压缩程度。Q4 比 Q8 小一半，质量略降但差别不大
_P	更细的量化变体，影响不大，不用管
.gguf	文件格式后缀

量化：为什么要压缩

原始模型每个数字都很精确，占空间大。压缩就是「把数字写得糙一点」——比如 0.1415926 记成 0.14。精度掉了，但文件小了一大半，跑起来也快。

压缩级别	35B 模型大概大小	适合
Q8	37 GB	内存很富裕，追求质量
Q4	23 GB	大多数人的选择，质量够用
Q2 / Q1	12 GB 以下	内存紧张，质量明显下降

token：AI 数数的单位

AI 不认「字」，它把文字切成小碎片来数，一个碎片叫一个 token。

中文里，一个汉字大约是 1 个 token；英文里一个单词通常 1～2 个。所以「你好，今天天气不错」大概 8 个 token。

> 为什么要在意 token？因为 AI 一次能看的 token 数量有上限，就像一个人一口气能记住的东西有限。这个上限叫「上下文窗口」。

上下文窗口：AI 一次能记住多少

假设窗口是 32K token，意思是：你发的所有内容 + AI 记得的历史对话 + AI 能用到的工具说明，加起来不能超过 32768 个 token。超了就报错。

这个上限在 App 的「设置」里能调，但要看你电脑内存够不够。第 5 章会详细讲。

第 2 章

准备工作

动手之前，先确认三件事：电脑够不够、软件装没装、模型下好啦没。

2.1 你的电脑能跑吗

点屏幕左上角苹果菜单 →「关于本机」，会弹出一个窗口。看两个地方：

项目	要求	在哪看
芯片	必须是 Apple Silicon（M1 / M2 / M3 / M4，或 Pro / Max / Ultra 变体）	关于本机里的「芯片」一行
内存	35B 的 Q4 模型建议 32 GB 以上；64 GB 更从容	关于本机里的「内存」一行
系统	macOS 14 或更高	关于本机里的「macOS」一行

> 如果是 Intel 芯片的老 Mac（芯片写着 Intel Core i5 / i7 / i9）：这个 App 跑不了。它只支持 Apple Silicon。

内存怎么估？拿 Q4 的 35B 模型举例，光模型就要占 23 GB，再加上对话历史和系统占用，实际会吃掉 25 GB 左右。所以 32 GB 的机器能跑，但最好别同时开很多别的程序。

2.2 下载模型文件

App 本身不带模型，你得自己下。常见的下载站：

- Hugging Face（huggingface.co）—— 国际站，模型最全
- ModelScope（modelscope.cn）—— 国内站，下载速度快
- unsloth 等量化作者的页面 —— 专门提供各种压缩版本

下载时注意找两种文件：

文件	必须要吗	作用
主模型 .gguf（体积最大的那个）	必须	就是模型本体，没有它跑不了

mmproj .gguf（文件名含 mmproj）	可选	让模型能看图。没有的话它只能读文字

> 重要：如果要下载视觉投影（mmproj），必须挑和主模型配套的那个。同一个模型页面下的 mmproj 才通用。用错了会在启动时报错，错误信息里会出现 `n_embd` 两个数字对不上。

文件放哪？

放哪都行，App 只记录路径，不会复制文件。但建议放在一个固定的文件夹，或者外置硬盘的模型目录里，方便以后管理。

下载一个 23 GB 的文件要一段时间。下载完先别急着打开 App，确认文件完整（大小和网站上写的一致）再说。

2.3 确认 DeepSeek Harness 已安装

DSH 是你要把模型接进去的那个工作台。检查一下「应用程序」文件夹里有没有它。如果没有，先去装好——这个 App 负责的是「提供模型」，DSH 才是你实际聊天的界面。

> 两个东西的关系：
> Xianzhu DSH Connect（本 App） = 模型的服务员，负责把模型请出来待命；
> DeepSeek Harness = 你工作的地方，负责跟你对话。

第 3 章

五步上手

跟着做就行，每一步都写清楚了点哪里、看什么。

1 打开 App

在「应用程序」文件夹里双击 Xianzhu DSH Connect。第一次打开如果弹出「无法验证开发者」，右键点它 → 选「打开」→ 再点「打开」即可。

2 导入模型

打开后默认在「模型」页。点左上角的 「导入模型组…」按钮，在弹出的文件选择框里找到你的 .gguf 文件，选中，点「打开」。

如果想一次导入多个模型，按住 ⌘（Command） 键多选就行。

> 小技巧：如果模型和它的 mmproj 在同一个文件夹里，只选主模型那一个文件就够了，App 会自动把旁边的 mmproj 配上。

导入后会看到列表里出现一行，前面有个圆点、后面写着文件大小。点一下那一行，圆点变成实心，表示「选中了这个模型」。

3 启动服务

点窗口左下角的「▶ 启动服务」。

接下来会等一会儿——第一次加载模型要 30 秒到 2 分钟，因为要把几十 GB 的数据从硬盘读进内存。这期间窗口右上角会显示「加载中」。

等右上角的圆点变绿、文字变成「运行中 · 端口 8686」，就成功了。

> 怎么知道真的好了？
> 看「日志」那一页，最后几行应该出现 `listening on http://127.0.0.1:8686`。看到这行就是彻底就绪。

4 在 DeepSeek Harness 里选模型

切到 DeepSeek Harness
窗口，看对话输入框的右下角，有一个显示当前模型名字的下拉菜单。点开它。

列表里应该出现你刚才导入的模型（名字后面带 Q4 或 Q8 之类的字样）。选中它。

5 开始对话

在输入框打字，按回车发送。就这么简单。

> 验证一下：发一句「你好，你是什么模型？」
> 如果回答里提到 Qwen / 通义千问，说明走的是你本地这个模型，成功了。
> 如果回答说是 DeepSeek，说明模型没选对，回到第 4 步重选。

以后每次怎么用

1. 打开 Xianzhu DSH Connect
2. 点「启动服务」（模型会记住，不用重新导入）
3. 等圆点变绿
4. 切到 DeepSeek Harness 开始用

用完可以关掉 Xianzhu DSH
Connect，服务会一起停。也可以勾选左下角的「登录时自启服务」，开机自动跑。

第 4 章

界面逐个说

窗口上方有五个标签页，从左到右依次介绍。

模型

管理你导入的模型。模型会按文件夹自动分组，一个组里可以有多个模型（比如同一系列的不同压缩级别）。

控件	作用
导入模型组…	添加 .gguf 文件，可以一次选多个
组名输入框	给这一组建个名字。留空就用文件夹名
导入投影	单独补一个 mmproj 视觉文件
改组名	把整组重命名
删除	从列表移除（不删硬盘上的文件）

每一行显示三样东西：模型名、文件大小、有没有配对视觉投影（有的话显示 ■ `mmproj-xxx.gguf`，没有就写「纯文本」）。

设置

分两块：运行参数、工具提示词。

参数	建议值	说明
端口	8686	服务监听的端口号。改的话 DSH 那边也要跟着改
上下文长度	32768 或 131072	一次能看多少 token，越大越吃内存
GPU 层数	全部卸载到 GPU	让显卡干活，最快
线程数	自动	CPU 线程数，一般不用动
并发槽位	1	同时处理几个请求。保持 1，见下方说明
Flash Attention	开	省内存、加速，建议一直开

按显存自动收缩	开	内存不够时自动降参数，避免启动失败
保留思考内容	开	让模型保留推理过程

> 为什么并发槽位必须是 1?
>
> 设成 8 的时候，总上下文会被平均切成 8 份。你以为设了 128K，实际每个请求只能用 16K。
> 如果你发现「明明设了 128K，怎么还是不够用」，先检查这个数字。

访问 / 公网

控制别的设备能不能连到你的模型。

选项	含义
仅本机	只有这台 Mac 能用。最安全，推荐
局域网	同一个 Wi-Fi 下的设备都能连（需要密钥）
公网	外网也能连。需要另外配隧道工具，有安全风险

下面列出的 http://192.168.x.x:8686/v1
这些地址，就是别的设备该填的接口地址。点右边的图标可以复制。

日志

模型的实时运行输出。启动失败、回答变慢、上下文不够，原因基本都能在这里看到。遇到问题先来这里看最后几行。

教程

就是你现在看的这份。

第 5 章

为什么会「装不下」

这一章解释最常见的一类问题。如果你已经遇到过报错，这里能找到原因。

三样东西挤在一个窗口里

每次你发一句话，AI 收到的其实不只是你的话，而是三部分打包：

1. 系统提示——告诉 AI「你是谁、该怎么做事」的一段说明
2. 工具说明——AI
   能用的每个工具（读文件、跑命令、搜索……）都要连说明书一起发过去，它才知道怎么用
3. 聊天记录——你之前说过的话、AI 答过的话、工具返回的结果

这三部分加起来，不能超过上下文窗口。超了就报错：`exceed_context_size_error`。

工具说明有多占地方?

在装了逆向工程插件的 DSH 里，实测数据是这样的：

内容	token 数	占比
逆向工程工具（229 个）	68,552	91%
其他日常工具（30 个）	5,341	7%
系统提示	1,731	2%
合计	75,624	100%

> 也就是说，每次你只说一句「你好」，AI 其实要先读 7 万多 token 的准备工作，才轮到你那句话。
>
> 如果模型的窗口只有 32K，光准备工作就超了两倍，请求直接失败。

三个解决办法

办法一：换更小的模型，把窗口开大

Q4 压缩的模型只要 23 GB 内存，省下的空间可以用来开大上下文。同样一台 64 GB 的机器，Q8 只能开 32K，Q4 能开到 128K。质量差别不大，但可用性天差地别。

办法二：关掉用不上的工具

在「设置 → 工具提示词」里，把不用的组取消勾选。比如你不做逆向分析，把 REA 那组关掉，立刻省下 68K。

办法三：新开一个会话

聊天记录越积越多也会占地方。聊到很长之后，新开一个对话能清空历史。

> 一个贴心设计：工具开关只对本地模型生效。
> 你切回 DeepSeek
> 云端时，工具会自动全部恢复——因为云端窗口够大，裁掉反而损失能力。

第 6 章

把速度调快

如果你觉得回答太慢，按这几条依次排查。

先搞清楚：两段速度不一样

AI 回答分两个阶段，速度差很多：

阶段	在干什么	通常速度
预填充	读你的问题 + 历史 + 工具说明	快，但量大会明显卡
生成	一个字一个字往外蹦	稳定，35B 大约 40～55 字/秒

如果「等了很久才开始出字」，问题在预填充——通常是上下文太长。如果「出字很慢」，问题在生成速度——通常是没有走 GPU。

检查清单

1. GPU 层数是不是全部卸载了？设置里选「全部卸载到 GPU (99)」。如果退回 CPU，速度会掉到十分之一。
2. 显存是不是爆了？看「日志」页有没有 Metal 相关的报错。爆了的话把上下文调小，或者关掉视觉投影。
3. 上下文是不是设太大了？128K 很好，但每次请求都要处理更长的内容。日常聊天 32K 通常够用，快很多。
4. 是不是同时开了很多程序？模型会占 25 GB 内存，浏览器开几十个标签会互相抢。

立竿见影的几招

- 日常用 32K 上下文，只在处理长文档时才临时开到 128K
- 纯文字对话不需要看图，可以在模型列表里把 mmproj 留空，省 1 GB
- 一次只问一件事，别把十个需求堆在一条消息里
- 长对话及时新开会话，历史越短越快

关于温度等高级参数

设置页最下面有个「额外 llama-server 参数」输入框，给懂行的人调采样用的。新手不用碰，留空就行。一时好奇填了，清空重来即可。

第 7 章

遇到问题怎么办

按症状查。每条都写了原因和做法。

Q：点「启动服务」后一直没反应，圆点不变绿

A：模型太大，还在从硬盘加载。等 1～2 分钟。如果超过 3 分钟还没好，切到「日志」页看最后几行，通常有明确的报错。

Q：启动后立刻提示「已退出 (code 1)」

A：多半是模型文件有问题，或者 mmproj 和主模型不配套。看日志里有没有 `mismatch` 或 `failed to load` 字样。试着在模型列表里把 mmproj 去掉再启动。

Q：DSH 的模型下拉里找不到本地模型

A：DSH 需要重启才认新配置。按 ⌘Q 完全退出 DSH，再重新打开。还不行就说明配置文件没写成功，看附录的配置位置。

Q：对话时报「连接错误」

A：模型服务没在跑，或者端口对不上。回到本 App 确认圆点是绿的，并且端口和 DSH 里配的一致（默认都是 8686）。

Q：报错里出现 exceed_context_size_error

A：上下文不够用。见第 5 章——要么开大窗口，要么关掉工具组，要么新开一个会话。注意改完之后要新建会话才生效。

Q：回答速度突然变慢很多

A：内存被占满了，模型退回了 CPU 运算。关掉些别的程序，或者把上下文调小、换更小的量化。

Q：中文回答乱码或者变成方块

A：极少见。通常是模型本身的问题，换一个模型文件试试。

Q：导入模型时文件选择框里选不中

A：确认选的是 .gguf 文件，不是 .safetensors 或 .bin——那些是别的格式，这个 App 不支持。

Q：换了模型之后 DSH 里对不上

A：DSH 的模型列表是按启动时读到的配置生成的。在 App 里换模型后，需要重启 DSH 才能看到新模型。

还是不行?

按这个顺序收集信息，能快速定位问题：

1. 切到「日志」页，往下翻到最底部
2. 找到第一行带 `E`（错误）或 `error` 的内容，抄下来
3. 截图整个窗口（包含标题栏的模型名）
4. 附上你的芯片型号和内存大小

附录

文件都在哪

一般不需要动这些文件，知道位置即可。

文件	作用
config.json	App 的所有设置：模型、端口、上下文
models.json	已导入的模型组清单
llama/	内置的推理引擎，不要删
logs/	每次启动的运行日志

位置：`~/Library/Application Support/XianzhuDSHConnect/`

DSH 那边的配置

文件	作用
cordis.patch.yml	模型路由、插件、工具组开关的规则
tool-groups.json	工具组开关的当前状态

位置：`~/.dsh/`（profiles/desktop/ 下的那个是主配置）

隐私说明

本地模型全程在你的电脑里运算，断开网线也能用。你的对话内容不会发往任何服务器。

唯一的外发情况是：你在 DSH 里主动切换到云端模型（DeepSeek）时，内容才会发到 DeepSeek 的服务器。这个是 DSH 本身的行为，和本 App 无关。

> 用完了。
>
> 现在你应该能自己导入模型、启动服务、在 DSH
> 里对话了。剩下的就是在实际使用中慢慢摸索——遇到问题回来翻第 7 章。